TRACE-EUCLIDEAN: MATHEMATICA VERIFICATION

CODE, NUMERICAL TABLES AND EXACT FINITE CHECKS

This report records the actual results of `TraceEuclideanVerification.m`. It contains the four numerical tables corresponding to Section 4 of the accompanying paper, the finite classification outputs, all check outcomes, and the complete Wolfram Language source. The calculations use the repository's public extracted inputs.

Table 1: Recorded computation results.

Component	Actual result	Meaning
Section 4 grid	952 / 952 resolved	Exact rational interval comparisons
Displayed volume bounds	87 / 87 checked	Six significant digits, rounded upwards
Retained pairs	24 classic-integral; 63 integral	Necessary numerical conditions
Classification	115 PASS; 0 FAIL	Exact finite checks
Archived Voight rows	2,773 / 2,773 verified	Integrity of the supplied lists

Scope. Retained pairs satisfy necessary numerical conditions; their retention does not assert the existence of trace-Euclidean lattices. Checking every supplied field polynomial does not prove that the list of fields is complete or free of isomorphic duplicates. General mathematical arguments and Lean dependency statements are documented separately in `TRUST.md` and `THEOREM_INDEX.md`.

Run identification. Core run (UTC): 2026-10-20T06:15:10Z.
Archived-row run (UTC): 2026-10-20T06:15:32Z.
Mathematica 15.0.0 for Microsoft Windows (64-bit) (May 15, 2026).
Code SHA-256:
`e19b88aba2ba79573923cb6e95ba5d2a4db71b86689c2e2e63a7d1752a9eb79b`
Public input SHA-256:
`279e00f077d34db49fa00a04da1ca528904a0edb3255af6368dd6fd38259cb7e`

The hashes identify public computation files. The report source and final PDF hashes are recorded in `results/artifact_manifest.json`.

1. Reproduction and interpretation

Keep this directory with its parent `inputs/` directory. Run the following commands from `Trace-Euclidean`, using a licensed Mathematica or Wolfram Engine installation:

```
wolframscript -file mathematica/run.wls
wolframscript -file mathematica/run_voight.wls
```

The first command writes the table, interval and classification results. The second checks every archived field row and can take longer. Both commands return a nonzero exit status on a failed check. Missing output indicates an incomplete run.

In an interactive Mathematica session:

```
Get["mathematica/TraceEuclideanVerification.m"];
TraceEuclideanVerification`RunVerification[]
TraceEuclideanVerification`RunVoightIntegrity[]
```

An explicit output directory can be passed to either function. Loading the `.m` file does not start a computation, clear user symbols, change the current working directory or terminate the kernel.

To refresh the report from the checked results and compile its PDF, install XeLaTeX and `latexmk`, then run:

```
python -m pip install -r mathematica/requirements-report.txt
python mathematica/build_report.py
python tools/check_computational_report.py
```

The builder validates the result files and their provenance, updates only the marked generated blocks in this TeX file, and compiles the PDF. Text outside those blocks remains editable. The complete code appendix is read directly from the sibling `.m` file.

To compile the supplied TeX without rerunning any calculation, place the `.tex` and `.m` files in one directory and run there:

```
latexmk -xelatex -halt-on-error TraceEuclideanVerification.tex
```

This requires only those two source files and the standard TeX packages used in the preamble. The JSON/CSV files are needed to refresh the numerical content and validate the complete result manifest.

Meaning of the computational checks.

- Logarithms, displayed h values and the additional finite Gamma checks use 90-digit evaluation. High precision alone is not an interval proof.
- Grid signs, cutoff constants and volume rounding use exact rational intervals. The arithmetic interpretation of $\beta(d)$ remains a mathematical input.
- Gram matrices, roots, discriminants and Voronoi vertices use exact finite algebra in Mathematica. These checks trust its implementation.
- The general covering inequalities and field-list completeness have separate mathematical dependencies, described in the theorem index.

2. Section 4: formulas and exact decisions

Let n be the lattice rank and d the degree of the totally real field. Denote the input root-discriminant lower bound by $\beta(d)$ and set

$$V(n,d) = \frac{(\pi d)^{nd}}{\Gamma\left(\frac{nd}{2}+1\right)^2 \beta(d)^{nd}}.$$

The program evaluates $\log V$ to avoid numerical underflow. It retains a pair when $V(n,d)c^{nd} \geq 1$, where

$$c = \begin{cases} 1, & \text{in the classic-integral case, or if } n = 1, \\ 2, & \text{in the integral case with } n \geq 2. \end{cases}$$

The displayed functions are

$$h_{\text{classic}}(n,d) = \frac{\log V(n,d)}{n},$$
$$h_{\text{integral}}(n,d) = \frac{\log V(n,d)}{n} + d\log 2.$$

The displayed integral h value uses the second formula even at $n = 1$; the rank-one retention decision still uses $c = 1$.

The input bounds are

$$\beta(d) = \begin{cases} D_{\min}(d)^{\frac{1}{d}}, & 1 \leq d \leq 9, \\ 14, & d = 10, \\ \dfrac{14083}{1000}, & d = 11, \\ \dfrac{36347}{1000}\exp\left(-\dfrac{10667}{1000d}\right), & d \geq 12. \end{cases}$$

The discriminants $D_{\min}(d)$ are listed in Table 2.

Table 2: Input minimal discriminants in degrees one through nine.

d	$D_{\min}(d)$
1	1
2	5
3	49
4	725
5	14641
6	300125
7	20134393
8	282300416
9	9685993193

These values agree with the released verification data. The degree-eleven constant has a separate Lean proof in the repository. The numerical program does not discharge field-list completeness or the other general theorem dependencies recorded in `TRUST.md`.

Exact grid decisions. We bound $\arctan(1/5)$ and $\arctan(1/239)$ by 80-term alternating series. Machin's identity

$$\pi = 16\arctan\left(\frac{1}{5}\right) - 4\arctan\left(\frac{1}{239}\right)$$

gives rational bounds for π. The positive exponential series through term 180 is followed by a geometric upper bound for its tail. Endpoints are rounded outwards to a denominator of 10^{85}.

For even nd, $\Gamma(nd/2+1)$ is a factorial. For odd nd, the half-integer Gamma formula expresses its square as a rational multiple of π. Each value $V(n,d)$ is therefore enclosed by rational arithmetic. In every grid decision the whole interval lies on one side of the threshold. The exact endpoint comparisons also check

$$4\pi e < 35 \quad \text{and} \quad 36.347\exp\left(-\frac{10.667}{15}\right) > 2\pi e.$$

Together with the analytical covering inequalities, these constants restrict the search to $1 \le n \le 34$ and $1 \le d \le 14$. The finite Gamma checks do not establish an inequality for all real arguments. Smallest absolute log feasibility margin on the grid: 0.006928945928110577151 3.

Table S1. Classic-integral retained pairs. All retained pairs are included. The h column is rounded to six decimal places. The upper bound for V is rounded upwards to six significant digits and compared with the rational upper endpoint. The CSV contains full-precision values and exact rational display values.

Table S1: Classic-integral retained pairs (24 rows).

d	n	$h(n,d)$	Upper bound for $V(n,d)$
1	1	1.386294	4
1	2	1.144730	9.86961
1	3	0.954941	17.546
1	4	0.798156	24.3523
1	5	0.664340	27.7075
1	6	0.547477	26.7053
1	7	0.443662	22.3235
1	8	0.350216	16.4732
1	9	0.265216	10.8802
1	10	0.187232	6.50334
1	11	0.115173	3.54985
1	12	0.048188	1.78293
2	1	2.066316	7.89569
2	2	1.373169	15.5855
2	3	0.871810	13.6731
2	4	0.477289	6.7474
2	5	0.151320	2.13102
3	1	2.268840	9.66819
3	2	1.046447	8.10835
3	3	0.199664	1.82029
4	1	2.151631	8.59888
4	2	0.359872	2.05391
5	1	1.777311	5.91394
6	1	1.423463	4.15148

Table S2. Classic-integral maximum rank and degree. Here $N(d)$ is the maximum retained rank at degree d, and $D(n)$ is the maximum retained degree at rank n. A zero denotes an empty retained set. Every entry of both summary arrays is shown.

Table S2: Classic-integral maximum retained rank and degree.

d	$N(d)$	n	$D(n)$	n	$D(n)$
1	12	1	6	18	0
2	5	2	4	19	0
3	3	3	3	20	0
4	2	4	2	21	0
5	1	5	2	22	0
6	1	6	1	23	0
7	0	7	1	24	0
8	0	8	1	25	0
9	0	9	1	26	0
10	0	10	1	27	0
11	0	11	1	28	0
12	0	12	1	29	0
13	0	13	0	30	0
14	0	14	0	31	0
		15	0	32	0
		16	0	33	0
		17	0	34	0

Table S3. Integral retained pairs. The conventions are those of Table S1. The displayed integral $h(n, d)$ includes $d \log 2$ at rank one, while the rank-one retention criterion uses $c = 1$ as explained in Section 2.

Table S3: Integral retained pairs (63 rows).

d	n	$h(n, d)$	Upper bound for $V(n, d)$
1	1	2.079442	4
1	2	1.837877	9.86961
1	3	1.648088	17.546
1	4	1.491303	24.3523
1	5	1.357488	27.7075
1	6	1.240624	26.7053
1	7	1.136809	22.3235
1	8	1.043364	16.4732
1	9	0.958363	10.8802
1	10	0.880379	6.50334
1	11	0.808320	3.54985
1	12	0.741335	1.78293
1	13	0.678744	0.829245
1	14	0.619997	0.359118
1	15	0.564641	0.145499
1	16	0.512302	0.0553806
1	17	0.462661	0.0198757
1	18	0.415452	0.00674795
1	19	0.370443	0.00217358
1	20	0.327436	0.000665996
1	21	0.286258	0.000194579
1	22	0.246758	0.0000543233
1	23	0.208803	0.0000145212
1	24	0.172276	0.00000372326
1	25	0.137071	9.17233e-7
1	26	0.103095	2.17439e-7
1	27	0.070265	4.9672e-8
1	28	0.038504	1.09492e-8
1	29	0.007745	2.33172e-9
2	1	3.452611	7.89569
2	2	2.759463	15.5855
2	3	2.258104	13.6731
2	4	1.863584	6.7474
2	5	1.537614	2.13102
2	6	1.259527	0.467384
2	7	1.016850	0.0753125

Continued

d	n	$h(n,d)$	Upper bound for $V(n,d)$
2	8	0.801460	0.00929131
2	9	0.607760	0.000905695
2	10	0.431728	0.0000715108
2	11	0.270373	0.00000466634
2	12	0.121408	2.55861e-7
3	1	4.348282	9.66819
3	2	3.125888	8.10835
3	3	2.279105	1.82029
3	4	1.628022	0.164364
3	5	1.097941	0.00739092
3	6	0.650372	0.000188877
3	7	0.262790	0.00000300103
4	1	4.924220	8.59888
4	2	3.132460	2.05391
4	3	1.924347	0.0784946
4	4	1.008213	0.000860924
4	5	0.268749	0.00000365579
5	1	5.243047	5.91394
5	2	2.857502	0.296275
5	3	1.278816	0.00141481
5	4	0.092788	0.00000138226
6	1	5.582346	4.15148
6	2	2.586614	0.0430868
6	3	0.631313	0.0000253506
7	2	1.143409	0.000600804
8	2	1.275464	0.000195603
9	2	0.520141	0.0000107957

Table S4. Integral maximum rank and degree. The arrays have the same meaning as in Table S2 and include zero entries.

Table S4: Integral maximum retained rank and degree.

d	$N(d)$	n	$D(n)$	n	$D(n)$
1	29	1	6	18	1
2	12	2	9	19	1
3	7	3	6	20	1
4	5	4	5	21	1
5	4	5	4	22	1
6	3	6	3	23	1
7	2	7	3	24	1
8	2	8	2	25	1
9	2	9	2	26	1
10	0	10	2	27	1
11	0	11	2	28	1
12	0	12	2	29	1
13	0	13	1	30	0
14	0	14	1	31	0
		15	1	32	0
		16	1	33	0
		17	1	34	0

3. Finite rank-one classification

Write

$$q(x) = x^t G x \quad \text{and} \quad G = \begin{pmatrix} a & b \\ b & c \end{pmatrix}.$$

In this normalization the squared covering radius, named `rho` in the source, is

$$R(G) = \frac{ac(a + c - 2b)}{4(ac - b^2)}.$$

The enumeration uses $2 \le c \le a \le 7$, $0 \le b \le c/2$ and $R(G) < 2$. An independent exact Voronoi calculation intersects rational supporting half-planes and evaluates q at every retained vertex. Its search bound is computed from the parallelogram enclosed by the $\pm e_i$ constraints. This checks each enumerated instance independently of the radius formula. The maximum determinant in the following list is 21.

Table 3: All 22 Gram candidates and independent Voronoi checks.

ID	(a, b, c)	$\det G$	Exact maximum q	Constraints
1	[2, 0, 2]	4	1	8
2	[2, 1, 2]	3	2/3	18
3	[3, 0, 2]	6	5/4	8
4	[3, 1, 2]	5	9/10	12
5	[3, 0, 3]	9	3/2	8
6	[3, 1, 3]	8	9/8	8
7	[4, 0, 2]	8	3/2	8
8	[4, 1, 2]	7	8/7	12
9	[4, 0, 3]	12	7/4	8
10	[4, 1, 3]	11	15/11	8
11	[4, 1, 4]	15	8/5	8
12	[4, 2, 4]	12	4/3	18
13	[5, 0, 2]	10	7/4	8
14	[5, 1, 2]	9	25/18	12
15	[5, 1, 3]	14	45/28	8
16	[5, 1, 4]	19	35/19	8
17	[5, 2, 4]	16	25/16	14
18	[5, 2, 5]	21	25/14	8
19	[6, 1, 2]	11	18/11	12
20	[6, 1, 3]	17	63/34	8
21	[6, 2, 4]	20	9/5	12
22	[7, 1, 2]	13	49/26	12

The public input orders the diagonal coefficients canonically; Table 3 uses the computational ordering.

Nine field candidates and six survivors. Here m is the squarefree radicand and k the integer index parameter. The (h, k) pairs solve the finite trace/norm/index equations, including their parity constraint. Empty solution sets exclude three rows.

Table 4: Nine field candidates and their integer solutions.

m	Gram triple	Norm	$R(G)$	(h, k) solutions
2	[4, 0, 2]	1	3/2	[[0, 1]]
3	[4, 2, 4]	1	4/3	[[-1, 1], [0, 2], [1, 1]]
5	[3, 1, 2]	1	9/10	[[0, 1]]
5	[5, 0, 2]	2	7/4	[]
5	[4, 1, 4]	3	8/5	[]
5	[6, 2, 4]	4	9/5	[[0, 1]]
13	[7, 1, 2]	1	49/26	[[0, 1]]
17	[6, 1, 3]	1	63/34	[]
21	[5, 2, 5]	1	25/14	[[-1, 1], [1, 1]]

Constructed representatives. The columns give the radicand, coefficient α, integral basis, trace Gram triple and exact squared covering radius. Expressions in Table 5 retain Wolfram Language syntax.

Table 5: Six constructed representatives.

m	α	Basis	Gram triple	$R(G)$
2	1	{Sqrt[2], 1}	{4, 0, 2}	3/2
3	2 + Sqrt[3]	{1, -1 + Sqrt[3]}	{4, 2, 4}	4/3
5	1	{(1 + Sqrt[5])/2, 1}	{3, 1, 2}	9/10
5	2	{(1 + Sqrt[5])/2, 1}	{6, 2, 4}	9/5
13	1	{(1 + Sqrt[13])/2, 1}	{7, 1, 2}	49/26
21	(5 + Sqrt[21])/2	{3 + (-1 - Sqrt[21])/2, 1}	{5, 2, 5}	25/14

Each representative passes six checks: an integral unimodular basis, a totally positive integral coefficient, its trace Gram matrix, its determinant identity, the exact radius and the strict radius threshold. The remaining algebraic and elimination checks are listed in Appendix A.

The general Clark–Jagy rational-farthest-point input and general ideal-theoretic classification arguments do not follow from this finite test suite. Their formalization scope is described separately in the repository.

4. Archived Voight row integrity

The optional run checks the digests and row counts of the six archived public files. For every supplied polynomial it tests shape and monicity, irreducibility, total reality, the positive square index equation, and the actual field discriminant, recomputed independently in Mathematica.

Table 6: Checks of all 2,773 archived rows.

Degree	Rows	Irreducible	Totally real	Index square	Field disc.
5	674	674	674	674	674
6	827	827	827	827	827
7	301	301	301	301	301
8	164	164	164	164	164
9	15	15	15	15	15
10	792	792	792	792	792

The final four columns of Table 6 count passing rows. Individual outcomes follow the original input order in `results/voight-rows-DEGREE.json`; the defining polynomials remain in `inputs/voight/`.

These are counts of supplied rows, not counts of distinct isomorphism classes. They do not prove field-list completeness. In particular, the degree-ten table contains known fields; the completeness input used by the theorem has its own precise hypotheses.

Sources and cross-checks. The archived files accompany John Voight, *Enumeration of totally real number fields of bounded root discriminant*. Bounds, archive timestamps and file digests are listed in `inputs/voight/README.md`. The verification uses those public copies.

Independent comparison scripts are `checks/classification.wls` and `checks/voight_polynomial_integrity.wls`. The consolidated package preserves the 115 classification assertions and adds the Section 4 tables and interval calculations. The public Python checks provide a separate implementation.

The computational supplement builds on repository revision

`7f8ecd2c60b72437e6e958b60e40d2638237ee36`.

It does not change Lean theorem sources, toolchain pins or the numerical inputs used by the released Lean project. Mathematica's exact arithmetic and Lean's disclosed `native_decide` compiler dependency are different computational trust assumptions.

Troubleshooting. For a missing Wolfram executable or license, check the installation and activation. For missing inputs, restore the repository layout. For stale results or a hash mismatch, use the matching code and inputs and rerun the affected computations. If TeX compilation fails, check that XeLaTeX, `latexmk`, the Latin Modern fonts and the packages in the preamble are installed. Preserve the compiler log when investigating an error.

Appendix A. Complete check outcomes

Section 4 assertions.

Table 7: All 17 Section 4 assertions.

Assertion	Actual	Expected
`summary_complete`	True	True
`coarse_degree_cutoff_exact`	True	True
`coarse_rank_cutoff_exact`	True	True
`all_952_comparisons_separated`	True	True
`expected_counts`	True	True
`gamma_batir_lower_bound_on_grid_numeric`	True	True
`gamma_strict_consequence_on_grid_numeric`	True	True
`classic_published_pairs`	True	True
`classic_published_rank_bounds`	True	True
`classic_published_degree_bounds`	True	True
`classic_published_h_numeric_rounding`	True	True
`classic_published_volume_upper_exact`	True	True
`integral_published_pairs`	True	True
`integral_published_rank_bounds`	True	True
`integral_published_degree_bounds`	True	True
`integral_published_h_numeric_rounding`	True	True
`integral_published_volume_upper_exact`	True	True

Classification: all 115 assertions.

Table 8: All 115 classification assertions.

ID	Assertion	Actual	Expected
1	`CJ_coefficient_order_and_formula`	True	True
2	`CJ_lower_bound_identity`	True	True
3	`CJ_polarization_normalization`	True	True
4	`integer_upper_bound_A_7`	True	True
5	`ideal_index_norm_algebra`	True	True
6	`quadratic_trace_norm_algebra`	True	True
7	`exactly_22_reduced_Gram_triples`	True	True
8	`published_22_reduced_Gram_triples`	True	True
9	`maximum_determinant_is_21`	True	True
10	`Gram_1_exact_Voronoi`	True	True
11	`Gram_1_rational_deep_holes`	True	True
12	`Gram_2_exact_Voronoi`	True	True
13	`Gram_2_rational_deep_holes`	True	True
14	`Gram_3_exact_Voronoi`	True	True
15	`Gram_3_rational_deep_holes`	True	True
16	`Gram_4_exact_Voronoi`	True	True

Continued

ID	Assertion	Actual	Expected
17	`Gram_4_rational_deep_holes`	True	True
18	`Gram_5_exact_Voronoi`	True	True
19	`Gram_5_rational_deep_holes`	True	True
20	`Gram_6_exact_Voronoi`	True	True
21	`Gram_6_rational_deep_holes`	True	True
22	`Gram_7_exact_Voronoi`	True	True
23	`Gram_7_rational_deep_holes`	True	True
24	`Gram_8_exact_Voronoi`	True	True
25	`Gram_8_rational_deep_holes`	True	True
26	`Gram_9_exact_Voronoi`	True	True
27	`Gram_9_rational_deep_holes`	True	True
28	`Gram_10_exact_Voronoi`	True	True
29	`Gram_10_rational_deep_holes`	True	True
30	`Gram_11_exact_Voronoi`	True	True
31	`Gram_11_rational_deep_holes`	True	True
32	`Gram_12_exact_Voronoi`	True	True
33	`Gram_12_rational_deep_holes`	True	True
34	`Gram_13_exact_Voronoi`	True	True
35	`Gram_13_rational_deep_holes`	True	True
36	`Gram_14_exact_Voronoi`	True	True
37	`Gram_14_rational_deep_holes`	True	True
38	`Gram_15_exact_Voronoi`	True	True
39	`Gram_15_rational_deep_holes`	True	True
40	`Gram_16_exact_Voronoi`	True	True
41	`Gram_16_rational_deep_holes`	True	True
42	`Gram_17_exact_Voronoi`	True	True
43	`Gram_17_rational_deep_holes`	True	True
44	`Gram_18_exact_Voronoi`	True	True
45	`Gram_18_rational_deep_holes`	True	True
46	`Gram_19_exact_Voronoi`	True	True
47	`Gram_19_rational_deep_holes`	True	True
48	`Gram_20_exact_Voronoi`	True	True
49	`Gram_20_rational_deep_holes`	True	True
50	`Gram_21_exact_Voronoi`	True	True
51	`Gram_21_rational_deep_holes`	True	True
52	`Gram_22_exact_Voronoi`	True	True
53	`Gram_22_rational_deep_holes`	True	True
54	`published_nine_field_Gram_candidates`	True	True
55	`exactly_nine_field_Gram_candidates`	True	True
56	`all_fundamental_discriminants_are_exhausted`	True	True
57	`candidate_1_all_trace_norm_index_solutions`	True	True
58	`candidate_2_all_trace_norm_index_solutions`	True	True
59	`candidate_3_all_trace_norm_index_solutions`	True	True

Continued

ID	Assertion	Actual	Expected
60	candidate_4_all_trace_norm_index_solutions	True	True
61	candidate_5_all_trace_norm_index_solutions	True	True
62	candidate_6_all_trace_norm_index_solutions	True	True
63	candidate_7_all_trace_norm_index_solutions	True	True
64	candidate_8_all_trace_norm_index_solutions	True	True
65	candidate_9_all_trace_norm_index_solutions	True	True
66	all_possible_b2_values	True	True
67	three_empty_rows_are_excluded	True	True
68	six_Gram_candidates_survive	True	True
69	all_survivors_except_m3_have_index_one	True	True
70	m3_both_values_two_force_cross_term_plus_or_minus_two	True	True
71	m3_forced_cross_traces_contradict_B_2	True	True
72	m3_other_b_values_have_norm_one	True	True
73	m3_conjugate_coefficients_are_unit_square_equivalent	True	True
74	m21_conjugate_coefficients_are_unit_square_equivalent	True	True
75	published_six_representative_Gram_and_radius_records	True	True
76	representative_1_unimodular_integral_basis	True	True
77	representative_1_positive_integral_coefficient	True	True
78	representative_1_trace_Gram	True	True
79	representative_1_determinant_identity	True	True
80	representative_1_exact_covering_radius	True	True
81	representative_1_strict_threshold	True	True
82	representative_2_unimodular_integral_basis	True	True
83	representative_2_positive_integral_coefficient	True	True
84	representative_2_trace_Gram	True	True
85	representative_2_determinant_identity	True	True
86	representative_2_exact_covering_radius	True	True
87	representative_2_strict_threshold	True	True
88	representative_3_unimodular_integral_basis	True	True
89	representative_3_positive_integral_coefficient	True	True
90	representative_3_trace_Gram	True	True
91	representative_3_determinant_identity	True	True
92	representative_3_exact_covering_radius	True	True
93	representative_3_strict_threshold	True	True
94	representative_4_unimodular_integral_basis	True	True
95	representative_4_positive_integral_coefficient	True	True
96	representative_4_trace_Gram	True	True
97	representative_4_determinant_identity	True	True
98	representative_4_exact_covering_radius	True	True
99	representative_4_strict_threshold	True	True
100	representative_5_unimodular_integral_basis	True	True
101	representative_5_positive_integral_coefficient	True	True
102	representative_5_trace_Gram	True	True

Continued

ID	Assertion	Actual	Expected
103	`representative_5_determinant_identity`	True	True
104	`representative_5_exact_covering_radius`	True	True
105	`representative_5_strict_threshold`	True	True
106	`representative_6_unimodular_integral_basis`	True	True
107	`representative_6_positive_integral_coefficient`	True	True
108	`representative_6_trace_Gram`	True	True
109	`representative_6_determinant_identity`	True	True
110	`representative_6_exact_covering_radius`	True	True
111	`representative_6_strict_threshold`	True	True
112	`constructed_Gram_matrices_are_exactly_the_six_survivors`	True	True
113	`m5_two_F_isometry_classes_are_distinct`	True	True
114	`Corollary_1_9_fields`	True	True
115	`m3_twisted_and_standard_radii`	True	True

Appendix B. Complete executable .m source

The listing is read directly from the file whose SHA-256 digest appears in the run identification. Line numbers count every source line, including blank lines; wrapped continuations have no additional number.

```
(* Trace-Euclidean: reproducible Mathematica computations.
   Load this file with Get, then call RunVerification[].
   RunVoightIntegrity[] is the separate, longer archived-row check.
   All paths are relative to this file. Loading it does not run checks,
   change the working directory, or clear the user's symbols.
   The accompanying PDF records a run of this exact source file. *)
BeginPackage["TraceEuclideanVerification`"];
RunVerification::usage =
  "RunVerification[outputDirectory] checks Section 4 and the classification.";
RunVoightIntegrity::usage =
  "RunVoightIntegrity[outputDirectory] checks all 2773 archived Voight rows.";
Begin["`Private`"];
baseDirectory = DirectoryName[$InputFileName];
packageDirectory = ParentDirectory[baseDirectory];
codeFile = FileNameJoin[{baseDirectory, "TraceEuclideanVerification.m"}];
inputFile = FileNameJoin[{packageDirectory, "inputs", "verification_inputs.json"}];
sha[path_] := IntegerString[FileHash[path, "SHA256"], 16, 64];
text[x_] := ToString[x, InputForm];
fail[name_] := Throw[Failure["Verification", <|"check" -> name|>], failureTag];
prepare[dir_] := (
  outputDirectory = If[dir === Automatic,
    FileNameJoin[{baseDirectory, "results"}], ExpandFileName[dir]];
  If[!DirectoryQ[outputDirectory],
    CreateDirectory[outputDirectory, CreateIntermediateDirectories -> True]];
  If[!DirectoryQ[outputDirectory], fail["output_directory"]];
);
writeJSON[name_, data_] := If[
  Export[FileNameJoin[{outputDirectory, name}], data, "RawJSON"] === $Failed,
  fail["export_" <> name]];
provenance[] := <|"tool" -> $Version, "code_sha256" -> sha[codeFile],
  "input_sha256" -> sha[inputFile],
  "run_utc" -> DateString[Now, {"ISODate", "T", "Time", "Z"}, TimeZone -> 0]|>;

(* Section 4: numerical evaluation and exact rational sign certificates.
   beta(d) is an arithmetic input. These computations do not prove its
   number-field lower-bound interpretation or the existence of retained pairs. *)
minDiscriminants =
  {1, 5, 49, 725, 14641, 300125, 20134393, 282300416, 9685993193};
beta[d_Integer] := Which[
  d <= 9, minDiscriminants[[d]]^(1/d), d == 10, 14,
  d == 11, 14083/1000, True, (36347/1000) Exp[-10667/(1000 d)]];
logVolume[n_Integer, d_Integer] :=
  n d Log[Pi d] - 2 LogGamma[n d/2 + 1] - n d Log[beta[d]];
effectiveC[case_, n_Integer] := If[case == "classic" || n == 1, 1, 2];
decimalString[x_] := text[N[x, 60]];
(* Only integer/rational strings from the public JSON input are accepted. *)
readRational[s_String] := Module[{parts},
  If[!StringMatchQ[s, RegularExpression["-?[0-9]+(/[0-9]+)?"]],
    fail["rational_input"]];
  parts = FromDigits /@ StringSplit[StringTrim[s, "-"], "/"];
```

```
  If[StringStartsQ[s, "-"], -1, 1] *
    If[Length[parts] == 1, First[parts], parts[[1]]/parts[[2]]]
];
snap[{lo_, hi_}] := {Floor[10^85 lo], Ceiling[10^85 hi]}/10^85;
atanBounds[q_] := Module[{lo},
  lo = Sum[(-1)^j/((2 j + 1) q^(2 j + 1)), {j, 0, 79}];
  {lo, lo + 1/(161 q^161)}
];
expBounds[x_] := Module[{s = 1, t = 1, j},
  If[!TrueQ[0 <= x < 182], fail["exponential_domain"]];
  Do[t = t x/j; s += t, {j, 1, 180}];
  snap[{s, s + (t x/181)/(1 - x/182)}]
];
(* Alternating arctangent bounds and Machin's identity enclose pi.
   The exponential tail is bounded by a geometric series after term 180.
   Every operation in these interval routines is rational arithmetic. *)
initializeIntervals[] := Module[{p = atanBounds[5], q = atanBounds[239]},
  piBounds = snap[{16 p[[1]] - 4 q[[2]], 16 p[[2]] - 4 q[[1]]}];
  eBounds = expBounds[1];
  expBoundsMain = expBounds[10667/1000];
  expBoundsCutoff = expBounds[10667/15000];
];
volumeInterval[n_, d_] := Module[{m = n d, k, coefficient, power},
  k = Floor[m/2];
  If[EvenQ[m],
    coefficient = d^m/(k!)^2; power = m,
    coefficient = d^m 16^(k + 1) ((k + 1)!)^2/((2 k + 2)!)^2;
    power = m - 1];
  coefficient /= Which[d <= 9, minDiscriminants[[d]]^n,
    d == 10, 14^m, d == 11, (14083/1000)^m,
    True, (36347/1000)^m];
  coefficient piBounds^power If[d >= 12, expBoundsMain^n, {1, 1}]
];
upperSix[x_] := Module[{k, unit},
  k = IntegerLength[Numerator[x]] - IntegerLength[Denominator[x]];
  If[x < 10^k, k--];
  unit = 10^(k - 5);
  Ceiling[x/unit] unit
];
sectionFour[] := Module[
  {workingPrecision = 90, rows = {}, grid = {}, certs = {}, summary,
   checks, n, d, case, c, lv, margin, keep, row, caseRows, bounds,
   threshold, decision, upper, published, selected, data, certificate},
  initializeIntervals[];
  Do[
    c = effectiveC[case, n];
    lv = N[logVolume[n, d], workingPrecision];
    margin = N[logVolume[n, d] + n d Log[c], workingPrecision];
    keep = TrueQ[margin >= 0];
    bounds = volumeInterval[n, d];
    threshold = If[case == "classic" || n == 1, 1, 2^(-n d)];
    decision = Which[bounds[[1]] >= threshold, True,
      bounds[[2]] < threshold, False, True, fail["unresolved_interval"]];
    If[decision =!= keep, fail["numeric_interval_disagreement"]];
    row = <|"case" -> case, "n" -> n, "d" -> d,
      "log_volume_bound" -> decimalString[lv],
```

```
      "h_new" -> decimalString[lv/n + If[case == "integral", d Log[2], 0]],
      "volume_bound" -> decimalString[Exp[lv]],
      "beta" -> decimalString[beta[d]],
      "log_feasibility_margin" -> decimalString[margin], "retained" -> keep|>;
    AppendTo[grid, row];
    AppendTo[certs, <|"case" -> case, "n" -> n, "d" -> d,
      "retained" -> decision, "separated" -> True|>];
    If[keep,
      upper = upperSix[bounds[[2]]];
      AssociateTo[row, "upper_six_rational" -> text[upper]];
      AppendTo[rows, row]],
    {case, {"classic", "integral"}}, {d, 1, 14}, {n, 1, 34}];
  summary = Association@Table[
    caseRows = Select[rows, #["case"] == case &];
    case -> <|"count" -> Length[caseRows],
      "N_d" -> Table[Max[Prepend[
        (#["n"] & /@ Select[caseRows, #["d"] == d &]), 0]], {d, 1, 14}],
      "D_n" -> Table[Max[Prepend[
        (#["d"] & /@ Select[caseRows, #["n"] == n &]), 0]], {n, 1, 34}]|>,
    {case, {"classic", "integral"}}];
  checks = <|
    "summary_complete" -> FreeQ[summary, _Missing],
    "coarse_degree_cutoff_exact" ->
      ((36347/1000)/expBoundsCutoff[[2]] > 2 piBounds[[2]] eBounds[[2]]),
    "coarse_rank_cutoff_exact" -> (4 piBounds[[2]] eBounds[[2]] < 35),
    "all_952_comparisons_separated" -> (Length[certs] == 952),
    "expected_counts" -> (summary["classic"]["count"] == 24 &&
      summary["integral"]["count"] == 63),
    "gamma_batir_lower_bound_on_grid_numeric" -> And @@ Table[TrueQ[
      N[LogGamma[m/2+1] -
        (Log[2]/2-m/2+(m/2+1/2) Log[m/2+1/2]), 70] >= 0], {m, 1, 476}],
    "gamma_strict_consequence_on_grid_numeric" -> And @@ Table[TrueQ[
      N[Log[m+1]/2+(m/2) Log[1+1/m], 70] > 0], {m, 1, 476}]|>;
  Do[
    selected = Select[rows, #["case"] == case &];
    published = publicInput["table_rows"][case];
    AssociateTo[checks, {
      case <> "_published_pairs" ->
        (Lookup[selected, {"n", "d"}] == Lookup[published, {"n", "d"}]),
      case <> "_published_rank_bounds" ->
        (summary[case]["N_d"] == publicInput["rank_bounds_by_degree"][case]),
      case <> "_published_degree_bounds" ->
        (summary[case]["D_n"] == publicInput["degree_bounds_by_rank"][case]),
      case <> "_published_h_numeric_rounding" -> And @@ MapThread[
        (Abs[ToExpression[#1["h_new"]] -
          readRational[#2["h_display"]]] <= 1/2000000) &,
        {selected, published}],
      case <> "_published_volume_upper_exact" -> And @@ MapThread[
        (readRational[#1["upper_six_rational"]] ==
          readRational[#2["volume_upper_display"]]) &, {selected, published}]}],
    {case, {"classic", "integral"}}];
  certificate = <|
    "method" -> "Rational intervals; Machin identity; series tail bounds; half-integer Gamma values.",
    "pi_interval" -> (text /@ piBounds),
    "e_interval" -> (text /@ eBounds),
    "exp_main_interval" -> (text /@ expBoundsMain),
```

```
    "exp_cutoff_interval" -> (text /@ expBoundsCutoff),
    "comparisons" -> Length[certs], "unresolved_comparisons" -> 0,
    "classic_retained" -> summary["classic"]["count"],
    "integral_retained" -> summary["integral"]["count"],
    "upward_rounded_values" -> Length[rows], "decisions" -> certs|>;
  data = <|"provenance" -> provenance[],
    "working_precision" -> workingPrecision,
    "minimal_discriminants" -> minDiscriminants,
    "summary" -> summary, "checks" -> checks, "rows" -> rows, "grid" -> grid,
    "all_passed" -> And @@ (TrueQ /@ Values[checks])|>;
  writeJSON["section4_tables.json", data];
  writeJSON["rational_certificate.json", certificate];
  KeyValueMap[Print[If[TrueQ[#2], "PASS: ", "FAIL: "], #1] &, checks];
  If[!TrueQ[data["all_passed"]], fail["section4_checks"]];
  Print["SECTION 4: 952 certified decisions; 24 / 63 retained pairs."];
  data
];

(* The following classification calculations preserve the released
   115 assertions and the independent finite Voronoi computation. *)
classification[] := Module[{},
checks = <||>;
checkRecords = {};
truth[x_] := TrueQ[FullSimplify[x]];
SetAttributes[check, HoldRest];
check[name_, statement_] := Module[{value = truth[statement]},
  AssociateTo[checks, name -> value];
  AppendTo[checkRecords,
    <|"id" -> name, "expected" -> True, "actual" -> value, "pass" -> value|>]
];
squareFreeIntegerQ[k_Integer] := And @@ (Last[#] == 1 & /@ FactorInteger[k]);
fundamentalDiscriminant[d_] := If[Mod[d, 4] == 1, d, 4 d];
omega[d_] := If[Mod[d, 4] == 1, (1 + Sqrt[d])/2, Sqrt[d]];
conjugate[d_, x_] := FullSimplify[x /.
  ((Power[base_Integer, exponent_Rational] /; base == d && Denominator[exponent] == 2) :>
    -base^exponent)];
fieldTrace[d_, x_] := FullSimplify[x + conjugate[d, x]];
fieldNorm[d_, x_] := FullSimplify[x conjugate[d, x]];
gram[t_] := {{t[[1]], t[[2]]}, {t[[2]], t[[3]]}};
canonicalGram[t_] := {Min[t[[1]], t[[3]]], t[[2]], Max[t[[1]], t[[3]]]} ;
rho[t_] := With[{a0 = t[[1]], b0 = t[[2]], c0 = t[[3]]},
  a0 c0 (a0 + c0 - 2 b0)/(4 (a0 c0 - b0^2))];
cjE[a0_, b0_, c0_] := a0 c0 (a0 - b0 + c0)/(4 a0 c0 - b0^2);
check["CJ_coefficient_order_and_formula", cjE[cc, 2 bb, aa] == rho[{aa, bb, cc}]];
check["CJ_lower_bound_identity", rho[{aa, bb, cc}] - aa/4 == aa (cc - bb)^2/(4 (aa cc - bb^2))];
check["CJ_polarization_normalization",
  Expand[({x1+y1,x2+y2}.{{aa,bb},{bb,cc}}.{x1+y1,x2+y2}) -
    ({x1,x2}.{{aa,bb},{bb,cc}}.{x1,x2}) - ({y1,y2}.{{aa,bb},{bb,cc}}.{y1,y2})] ==
    2 {x1,x2}.{{aa,bb},{bb,cc}}.{y1,y2}];
check["integer_upper_bound_A_7", Reduce[aa >= 2 && aa/4 < 2, aa, Integers] ==
  (aa == 2 || aa == 3 || aa == 4 || aa == 5 || aa == 6 || aa == 7)];
check["ideal_index_norm_algebra", na nb^2 == (na ni^2) (nb/ni)^2];
check["quadratic_trace_norm_algebra",
  ((cc + hh Sqrt[dd])/2) ((cc - hh Sqrt[dd])/2) == (cc^2 - dd hh^2)/4];
(* Independent exact Voronoi computation. The constraints from +/-e_i
   enclose the cell in a parallelogram on which q <= boundRadius.
```

```
    Cauchy-Schwarz excludes any further constraint q(v)>4*boundRadius.
    For 2x2 positive definite g, lambda_min(g) >= det(g)/tr(g), so the
    integer-vector search below includes every vector that can matter. *)
certifiedVoronoi[g_] := Module[
  {corners, boundRadius, coordinateBound, vectors, rows, bounds, candidates, vertex, ids, vertices},
  corners = LinearSolve[g, # Diagonal[g]/2] & /@ Tuples[{-1, 1}, 2];
  boundRadius = Max[(# . g . #) & /@ corners];
  coordinateBound = Ceiling[Sqrt[4 boundRadius Tr[g]/Det[g]]];
  vectors = Select[Tuples[Range[-coordinateBound, coordinateBound], 2],
    0 < # . g . # <= 4 boundRadius &];
  rows = (# . g) & /@ vectors;
  bounds = (# . g . #/2) & /@ vectors;
  candidates = Reap[
    Do[
      If[Det[rows[[ids]]] != 0,
        vertex = LinearSolve[rows[[ids]], bounds[[ids]]];
        If[And @@ Thread[rows . vertex <= bounds], Sow[vertex]]
      ],
      {ids, Subsets[Range[Length[vectors]], {2}]}
    ]
  ][[2]];
  vertices = DeleteDuplicates[First[candidates]];
  <|"gram" -> g, "constraint_norm_bound" -> 4 boundRadius,
    "coordinate_bound" -> coordinateBound, "constraint_count" -> Length[vectors],
    "vertices" -> vertices,
    "maximum" -> Max[(# . g . #) & /@ vertices]|>
];

(* Exhaustive reduced integral Gram matrices under the proven A <= 7 bound. *)
gramTriples = Reap[Do[
  If[a0 c0 (a0+c0-2 b0) < 8 (a0 c0-b0^2), Sow[{a0,b0,c0}]],
  {a0,2,7}, {c0,2,a0}, {b0,0,Floor[c0/2]}]][[2,1]];
check["exactly_22_reduced_Gram_triples", Length[gramTriples] == 22];
check["published_22_reduced_Gram_triples",
  Sort[canonicalGram /@ gramTriples] ==
    Sort[publicInput["reduced_gram_triples"]]];
check["maximum_determinant_is_21", Max[Det[gram[#]] & /@ gramTriples] == 21];
gramCertificates = Association @ Table[With[{t = gramTriples[[i]]},
  Print["Independent Voronoi computation ", i, "/22: ", t];
  ToString[i] -> certifiedVoronoi[gram[t]]], {i,Length[gramTriples]}];
Do[With[{data = gramCertificates[ToString[i]], t = gramTriples[[i]]},
  check["Gram_" <> ToString[i] <> "_exact_Voronoi", data["maximum"] == rho[t]];
  check["Gram_" <> ToString[i] <> "_rational_deep_holes", MatrixQ[data["vertices"],
    (IntegerQ[#] || Head[#] === Rational) &]];
], {i,Length[gramTriples]}];

fieldRows = Reap[Do[With[{delta = fundamentalDiscriminant[d], det = Det[gram[t]]},
  If[squareFreeIntegerQ[d] && Mod[det,delta] == 0 &&
     (Mod[d,4] == 1 || And @@ (EvenQ /@ t)),
    Sow[{d,t,det/delta,rho[t]}]]],
  {t,gramTriples}, {d,2,Det[gram[t]]}]][[2,1]];
fieldRows = SortBy[fieldRows, {#[[1]],#[[3]],#[[2]]} &];
check["published_nine_field_Gram_candidates",
  Sort[(Join[{#[[1]]}, canonicalGram[#[[2]]], {#[[3]]}] & /@ fieldRows)] ==
    Sort[publicInput["field_gram_candidates"]]];
expectedRows = {
```

```
  {2,{4,0,2},1,3/2}, {3,{4,2,4},1,4/3},
  {5,{3,1,2},1,9/10}, {5,{5,0,2},2,7/4},
  {5,{4,1,4},3,8/5}, {5,{6,2,4},4,9/5},
  {13,{7,1,2},1,49/26}, {17,{6,1,3},1,63/34},
  {21,{5,2,5},1,25/14}
};
check["exactly_nine_field_Gram_candidates", fieldRows == expectedRows];
check["all_fundamental_discriminants_are_exhausted",
  DeleteDuplicates[fundamentalDiscriminant /@ fieldRows[[All,1]]] == {8,12,5,13,17,21}];
traceNormSolutions[row_] := Module[
  {d = row[[1]], c0 = row[[2,3]], n0 = row[[3]], delta, hmax, kmax},
  delta = fundamentalDiscriminant[d];
  hmax = Floor[Sqrt[c0^2/delta]]; kmax = Floor[Sqrt[c0^2/(4 n0)]];
  Select[Tuples[{Range[-hmax,hmax], Range[1,kmax]}],
    Mod[c0-delta #[[1]],2] == 0 && c0^2-delta #[[1]]^2 == 4 n0 #[[2]]^2 &]
];
solutions = traceNormSolutions /@ fieldRows;
expectedSolutions = {{{0,1}}, {{-1,1},{0,2},{1,1}}, {{0,1}}, {}, {},
  {{0,1}}, {{0,1}}, {}, {{-1,1},{1,1}}};
Do[check["candidate_" <> ToString[i] <> "_all_trace_norm_index_solutions",
  solutions[[i]] == expectedSolutions[[i]]], {i,Length[fieldRows]}];
possibleCoefficients = MapThread[Function[{row, sol},
  FullSimplify[(row[[2,3]]+#[[1]] Sqrt[fundamentalDiscriminant[row[[1]]]])/2] & /@ sol],
  {fieldRows,solutions}];
expectedCoefficients = {{1},{2-Sqrt[3],2,2+Sqrt[3]},{1},{},{},{2},{1},{},
  {(5-Sqrt[21])/2,(5+Sqrt[21])/2}};
check["all_possible_b2_values", possibleCoefficients == expectedCoefficients];
check["three_empty_rows_are_excluded", Flatten[Position[solutions,{},1]] == {4,5,8}];
survivingRows = Pick[fieldRows, (Length[#] > 0 & /@ solutions), True];
check["six_Gram_candidates_survive", Length[survivingRows] == 6];
check["all_survivors_except_m3_have_index_one",
  And @@ Flatten[MapThread[Function[{row,sol},
    If[row[[1]] == 3, {True}, (#[[2]] == 1 & /@ sol)]], {fieldRows,solutions}]]];
check["m3_both_values_two_force_cross_term_plus_or_minus_two",
  Reduce[tt^2 == 4,tt,Reals] == (tt == -2 || tt == 2)];
check["m3_forced_cross_traces_contradict_B_2",
  (fieldTrace[3,#] & /@ {-2,2}) == {-4,4} &&
  FreeQ[fieldTrace[3,#] & /@ {-2,2},2]];
check["m3_other_b_values_have_norm_one", fieldNorm[3,2-Sqrt[3]] == 1 &&
  fieldNorm[3,2+Sqrt[3]] == 1];
Do[With[{b = If[d == 3,2+Sqrt[3],(5+Sqrt[21])/2]},
  check["m" <> ToString[d] <> "_conjugate_coefficients_are_unit_square_equivalent",
    fieldNorm[d,b] == 1 && conjugate[d,b] == b conjugate[d,b]^2]],
  {d,{3,21}}];

(* Construct the six F-isometry representatives, independently of enumeration. *)
representatives = {
  {2,1,{Sqrt[2],1},{4,0,2},3/2},
  {3,2+Sqrt[3],{1,Sqrt[3]-1},{4,2,4},4/3},
  {5,1,{omega[5],1},{3,1,2},9/10},
  {5,2,{omega[5],1},{6,2,4},9/5},
  {13,1,{omega[13],1},{7,1,2},49/26},
  {21,(5+Sqrt[21])/2,{3-omega[21],1},{5,2,5},25/14}
};
check["published_six_representative_Gram_and_radius_records",
  Sort[({#[[1]], canonicalGram[#[[4]]], ToString[#[[5]], InputForm]} & /@
```

```
      representatives)] ==
    Sort[({#["m"], canonicalGram[#["gram"]], #["radius"]} & /@
      publicInput["representatives"])]];
Do[Module[{d = row[[1]], a0 = row[[2]], beta = row[[3]], u, g, tag},
  tag = "representative_" <> ToString[i];
  u = Transpose[Table[With[{q0 = FullSimplify[(x-conjugate[d,x])/(omega[d]-conjugate[d,omega[d]])]},
    {FullSimplify[x-q0 omega[d]],q0}], {x,beta}]];
  g = Table[fieldTrace[d,a0 x y], {x,beta}, {y,beta}];
  check[tag <> "_unimodular_integral_basis", MatrixQ[u,IntegerQ] && Abs[Det[u]] == 1];
  check[tag <> "_positive_integral_coefficient", a0 > 0 && conjugate[d,a0] > 0 &&
    IntegerQ[fieldTrace[d,a0]] && IntegerQ[fieldNorm[d,a0]]];
  check[tag <> "_trace_Gram", g == gram[row[[4]]]];
  check[tag <> "_determinant_identity", Det[g] == fundamentalDiscriminant[d] fieldNorm[d,a0]];
  check[tag <> "_exact_covering_radius", rho[row[[4]]] == row[[5]]];
  check[tag <> "_strict_threshold", row[[5]] < 2];
], {i,Length[representatives]}, {row,{representatives[[i]]}}];
check["constructed_Gram_matrices_are_exactly_the_six_survivors",
  ({#[[1]],#[[4]],fieldNorm[#[[1]],#[[2]]],#[[5]]} & /@ representatives) == survivingRows];
check["m5_two_F_isometry_classes_are_distinct", fieldNorm[5,1] == 1 && fieldNorm[5,2] == 4];
check["Corollary_1_9_fields", Select[{2,3,5,13,21},
  If[Mod[#,4] == 1,(#+1)^2/(8 #),(#+1)/2] < 2 &] == {2,5,13}];
check["m3_twisted_and_standard_radii", rho[{4,2,4}] == 4/3 && rho[{6,0,2}] == 2];

jsonData = MapThread[Function[{row,sol,coeff},
  <|"m" -> row[[1]],"triple" -> row[[2]],"norm" -> row[[3]],
    "rho" -> ToString[row[[4]],InputForm],"h_k_solutions" -> sol,
    "possible_b2" -> (ToString[#,InputForm] & /@ coeff)|>],
  {fieldRows,solutions,possibleCoefficients}];
writeJSON["gram_candidates.json", <|"all_22_triples" -> gramTriples,
  "maximum_determinant" -> Max[Det[gram[#]] & /@ gramTriples],
  "nine_field_candidates" -> jsonData|>];
writeJSON["classification_table.json", Map[ToString[#,InputForm] &, representatives,{2}]];
writeJSON["voronoi_certificates.json", Map[Function[data,Map[ToString[#,InputForm] &,data]],
    gramCertificates]];
KeyValueMap[Print[If[TrueQ[#2],"PASS: ","FAIL: "],#1] &,checks];
allPassed = And @@ (TrueQ /@ Values[checks]);
writeJSON["section6_exact_checks.json", <|"Mathematica_version" -> $Version,"arithmetic" -> "Exact
    symbolic, integer and rational arithmetic.",
  "scope" -> "Clark-Jagy normalization and lower-bound algebra; exhaustive reduced Gram matrices under A
    <=7; fundamental discriminant and trace-parity filters; trace/norm/index solutions; six constructions;
     independent finite Voronoi certificates. Rational attainment and the general ideal-index identity are
     justified in the paper and source audit, not inferred from tests.",
  "provenance" -> provenance[], "check_records" -> checkRecords,
  "checks" -> checks,"all_passed" -> allPassed|>];
Print["SUMMARY: ",Count[Values[checks],True]," PASS / ",Count[Values[checks],Except[True]]," FAIL"];
If[!allPassed || Length[checks] != 115, fail["classification_checks"]];
<|"count" -> Length[checks], "all_passed" -> allPassed|>
];


tableSpecifications = {
  <|"file" -> "5-17.txt", "degree" -> 5, "rows" -> 674,
    "lineRecords" -> False,
    "sha256" ->
      "7c2ce57f51de3ad34bb60ade85659689fc629406315e9699a144f737d1b34192"|>,
  <|"file" -> "6-16.txt", "degree" -> 6, "rows" -> 827,
```

```
    "lineRecords" -> False,
    "sha256" ->
      "6e6bc75b54ad2643a42a3dfdd9497347eb89735d00c5b7367e934ad32d8ae082"|>,
  <|"file" -> "7-15.5.txt", "degree" -> 7, "rows" -> 301,
    "lineRecords" -> False,
    "sha256" ->
      "95ed179560eb866445eee3da22826cf98cc5eb0bc7f5091c27f4d332dc421a3d"|>,
  <|"file" -> "8-15.txt", "degree" -> 8, "rows" -> 164,
    "lineRecords" -> False,
    "sha256" ->
      "204a0ea04e2f4079f6fea20369ad8822d4d678a718272b9aaea68cf144b83294"|>,
  <|"file" -> "9-14.5.txt", "degree" -> 9, "rows" -> 15,
    "lineRecords" -> False,
    "sha256" ->
      "9c14aa90371184020bb121c36e63ceaa5bd4ada0eb55198fd94d4df67d6d3821"|>,
  <|"file" -> "10.txt", "degree" -> 10, "rows" -> 792,
    "lineRecords" -> True,
    "sha256" ->
      "cceac525009d9ab6b947e17186a5e74df2643eace152955702d306af6defbe83"|>
};

toMathematicaExpression[text_] :=
  ToExpression[StringReplace[text, {"[" -> "{", "]" -> "}"}]];

readRows[specification_] := Module[{path, text, lines},
  path = FileNameJoin[{inputDirectory, specification["file"]}];
  text = Import[path, "Text"];
  If[!StringMatchQ[text, RegularExpression["[0-9,\\[\\]\\s-]+"]],
    fail["voight_input_characters"]];
  If[TrueQ[specification["lineRecords"]],
    lines = Select[StringSplit[text, {"\r\n", "\n"}],
      StringLength[StringTrim[#]] > 0 &];
    toMathematicaExpression /@ lines,
    toMathematicaExpression[text]
  ]
];

checkRow[row_, degree_] := Module[
  {fieldDiscriminant, coefficients, polynomial, polynomialDiscriminant,
    quotient, index, irreducible, totallyReal, fieldDiscriminantComputed},
  If[! ListQ[row] || Length[row] != 2 || ! IntegerQ[row[[1]]] ||
      ! ListQ[row[[2]]] || ! And @@ (IntegerQ /@ row[[2]]) ||
      Length[row[[2]]] != degree + 1 || Last[row[[2]]] != 1,
    Return[<|"shape" -> False|>]
  ];
  fieldDiscriminant = row[[1]];
  coefficients = row[[2]];
  polynomial = Sum[coefficients[[k + 1]] x^k, {k, 0, degree}];
  polynomialDiscriminant = Discriminant[polynomial, x];
  quotient = If[Divisible[polynomialDiscriminant, fieldDiscriminant],
    Quotient[polynomialDiscriminant, fieldDiscriminant], -1];
  index = If[quotient > 0 && IntegerQ[Sqrt[quotient]], Sqrt[quotient], -1];
  irreducible = TrueQ[IrreduciblePolynomialQ[polynomial]];
  totallyReal = TrueQ[CountRoots[polynomial, {x, -Infinity, Infinity}] == degree];
  fieldDiscriminantComputed = NumberFieldDiscriminant[
    Root[Function[z, Evaluate[polynomial /. x -> z]], 1]];
```

```
  <|
    "shape" -> True,
    "irreducible" -> irreducible,
    "totallyReal" -> totallyReal,
    "indexSquare" -> (index > 0),
    "fieldDiscriminant" ->
      TrueQ[fieldDiscriminantComputed == fieldDiscriminant],
    "index" -> index
  |>
];

checkTable[specification_] := Module[
  {path, hash, rows, checks, shapePass, irreduciblePass, indexPass,
    totallyRealPass, fieldPass, indices, failures},
  path = FileNameJoin[{inputDirectory, specification["file"]}];
  hash = IntegerString[FileHash[path, "SHA256"], 16, 64];
  If[hash != specification["sha256"],
    fail["voight_input_hash"]
  ];
  rows = readRows[specification];
  If[Length[rows] != specification["rows"],
    fail["voight_row_count"]
  ];
  checks = checkRow[#, specification["degree"]] & /@ rows;
  shapePass = Count[checks, association_ /; TrueQ[association["shape"]]];
  irreduciblePass =
    Count[checks, association_ /; TrueQ[association["irreducible"]]];
  totallyRealPass =
    Count[checks, association_ /; TrueQ[association["totallyReal"]]];
  indexPass = Count[checks, association_ /; TrueQ[association["indexSquare"]]];
  fieldPass =
    Count[checks, association_ /; TrueQ[association["fieldDiscriminant"]]];
  indices = Cases[checks, association_ /; IntegerQ[association["index"]] :>
    association["index"]];
  failures = Pick[Range[Length[checks]],
    Map[! (TrueQ[#["shape"]] && TrueQ[#["irreducible"]] &&
        TrueQ[#["totallyReal"]] && TrueQ[#["indexSquare"]] &&
        TrueQ[#["fieldDiscriminant"]]) &,
      checks]];
  writeJSON["voight-rows-" <> ToString[specification["degree"]] <> ".json", checks];
  Print[specification["file"], ": ", Length[rows], " rows, ",
    fieldPass, " field discriminants verified"];
  <|
    "file" -> specification["file"],
    "degree" -> specification["degree"],
    "sha256" -> hash,
    "rows" -> Length[rows],
    "shape_pass" -> shapePass,
    "irreducible_pass" -> irreduciblePass,
    "totally_real_pass" -> totallyRealPass,
    "polynomial_discriminant_square_index_pass" -> indexPass,
    "field_discriminant_pass" -> fieldPass,
    "maximum_index" -> Max[indices],
    "failed_rows" -> failures,
    "pass" -> (failures === {})
  |>
```

```
];


RunVerification[dir_: Automatic] := Catch[Block[{$HistoryLength = 0},
  prepare[dir];
  publicInput = Import[inputFile, "RawJSON"];
  If[!AssociationQ[publicInput] || publicInput["schema_version"] =!= 3,
    fail["public_input_schema"]];
  Print["Mathematica: ", $Version];
  sectionFourResult = sectionFour[];
  classificationResult = classification[];
  runResult = <|"provenance" -> provenance[],
    "section4_checks" -> Length[sectionFourResult["checks"]],
    "grid_comparisons" -> 952, "upward_rounded_values" -> 87,
    "classification_checks" -> classificationResult["count"],
    "all_passed" -> (TrueQ[sectionFourResult["all_passed"]] &&
      TrueQ[classificationResult["all_passed"]])|>;
  writeJSON["verification_summary.json", runResult];
  runResult
], failureTag];

(* Optional archived-row integrity check. No completeness or uniqueness
   of the field lists is inferred from these finite computations. *)
RunVoightIntegrity[dir_: Automatic] := Catch[Block[{$HistoryLength = 0},
  prepare[dir];
  inputDirectory = FileNameJoin[{packageDirectory, "inputs", "voight"}];
  tableResults = checkTable /@ tableSpecifications;
  voightResult = <|"provenance" -> provenance[], "tables" -> tableResults,
    "total_rows" -> Total[Lookup[tableResults, "rows"]],
    "irreducible_pass" -> Total[Lookup[tableResults, "irreducible_pass"]],
    "totally_real_pass" -> Total[Lookup[tableResults, "totally_real_pass"]],
    "polynomial_discriminant_square_index_pass" ->
      Total[Lookup[tableResults, "polynomial_discriminant_square_index_pass"]],
    "field_discriminant_pass" ->
      Total[Lookup[tableResults, "field_discriminant_pass"]],
    "pass" -> And @@ Lookup[tableResults, "pass"]|>;
  writeJSON["voight_integrity.json", voightResult];
  If[!TrueQ[voightResult["pass"]], fail["voight_checks"]];
  Print["VOIGHT: ", voightResult["total_rows"], " rows verified."];
  voightResult
], failureTag];
End[];
EndPackage[];
```